

DISEÑO Y ANÁLISIS TEMPORAL DE UNA UNIDAD ARITMÉTICO LÓGICA

PRÁCTICA 4



Introducción a los Computadores
1º Grado Ingeniería Informática

Natalia Ayuso Luis M. Ramos Juan Segarra Víctor Viñals
nayuso@unizar.es luisma@unizar.es jsegarra@unizar.es victor@unizar.es



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Departamento de
Informática e Ingeniería
de Sistemas
Universidad Zaragoza

1 RESUMEN

En esta práctica vamos a construir una Unidad Aritmético-Lógica (ALU) de 16 bits con Logisim. Además del resultado, la ALU calculará también varios indicadores (flags): cero, signo, acarreo y desbordamiento.

El diseño será modular. Esto significa construir bloques sencillos, con funciones específicas, y luego replicarlos y conectarlos para conseguir una función más compleja.

Finalmente realizaremos un análisis temporal de todo el diseño, para estimar la frecuencia máxima de cálculo que podría ofrecer la ALU.

Para poder realizar la práctica hay que llevar resuelto el Estudio Previo del Apartado 2.2. Al llegar al laboratorio coloca encima de la mesa el Estudio Previo de forma visible.

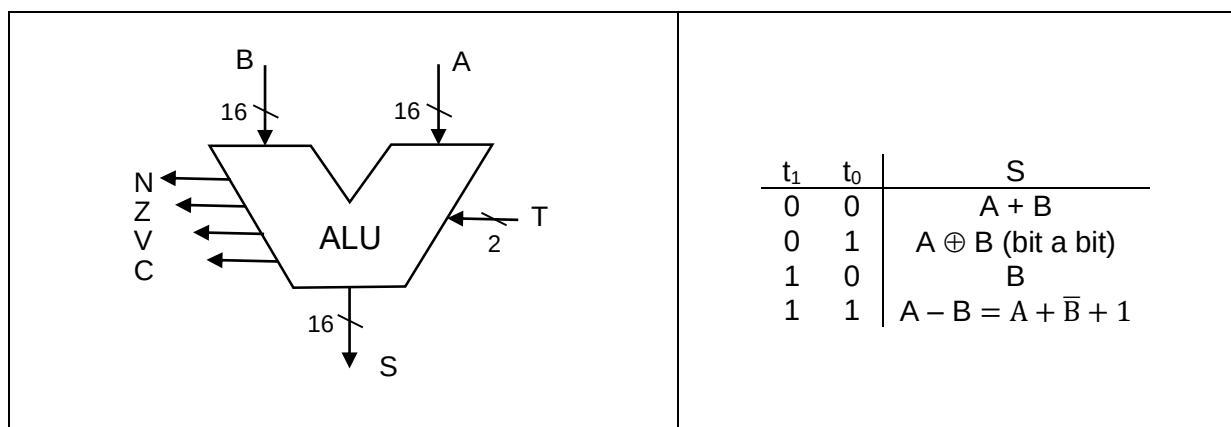
Cuando el circuito funcione correctamente debe **entregarse a través de Moodle**, junto con **los retardos obtenidos del análisis temporal del apartado 3**.

2 DISEÑO UNA UNIDAD ARITMÉTICO-LÓGICA (ALU)

2.1 ESPECIFICACIÓN DE FUNCIONAMIENTO DE LA ALU

La ALU a diseñar tiene dos vectores de entrada de 16 bits (**A** y **B**) y obtiene como resultado otro vector de 16 bits (**S**). Dependiendo del valor de la entrada de control **T**, la salida **S** corresponde a la operación especificada en la tabla.

$\underline{A} = (a_{15}, \dots, a_0)$ $\underline{B} = (b_{15}, \dots, b_0)$ $\underline{T} = (t_1, t_0)$ $\underline{S} = (s_{15}, \dots, s_0)$



Además, sobre el resultado se calculan los siguientes indicadores:

- *flag* de signo: indica el signo del resultado
- *flag* de cero: indica que el resultado es cero
- *flag* de desbordamiento de enteros:

Indica resultado incorrecto (no representable)

- *flag* de acarreo: último acarreo del sumador

$$\mathbf{N} = s_{15}$$

$$\mathbf{Z} = 1 \leftrightarrow S = 0$$

$$\mathbf{V} = \overline{a_{15}} \cdot \overline{b'_{15}} \cdot s_{15} + a_{15} \cdot b'_{15} \cdot \overline{s_{15}}$$

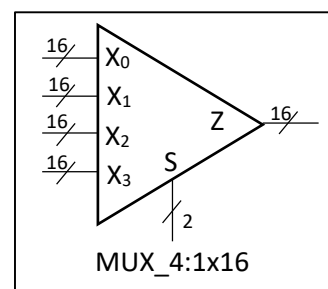
$$\text{Nota: } b'_{15} = \begin{cases} b_{15} & \text{si } t_1 = 0 \\ \overline{b_{15}} & \text{si } t_1 = 1 \end{cases}$$

$$\mathbf{C} = c_{16}$$

2.2 TRABAJO PREVIO: DISEÑO MODULAR

Antes de llegar al laboratorio diseña sobre papel los siguientes bloques:

- **MUX_4:1**: multiplexor de 4 entradas. Diseñalo usando puertas AND, OR y NOT.
- **MUX_4:1x4**: multiplexor de 4 entradas de 4 bits. Usa 4 **MUX_4:1**. Es muy importante que tengas claro este bloque combinacional. Dispone de 4 entradas de 4 bits y una salida de 4 bits. Con dos bits de selección se indica cuál de las cuatro entradas de 4 bits aparece en la salida de 4 bits.
- **MUX_4:1x16**: multiplexor de 4 entradas de 16 bits. Usa 4 **MUX_4:1x4**.
- **ALU**: unidad aritmético-lógica. Usa el **MUX_4:1x16**, un sumador de naturales de 16 bits (uno, no dos), puertas XOR y otras puertas para el cálculo de los *flags*.



2.3 TRABAJO EN EL LABORATORIO: CONSTRUCCIÓN Y SIMULACIÓN CON LOGISIM

- En una hoja nueva de Logisim añade un componente nuevo (*Proyecto / Añadir circuito*). Nómbralo **MUX4:1**. Añade cuatro entradas de un bit **x0**, **x1**, **x2** y **x3**. Añade dos entradas de un bit para la selección **s1** y **s0**. Añade una salida de un bit **z**. Dibuja tu diseño y comprueba su funcionamiento.
- Vuelve a la hoja principal (doble clic en **main**) y añade otro componente. Nómbralo **MUX_4:1x4**. Coloca 4 **MUX_4:1**. Coloca las entradas (**x0**, **x1**, **x2**, **x3**, **s1** y **s0**) y una salida **Z**. Define **x0**, **x1**, **x2**, **x3** y **Z** como entradas/salidas de 4 bits (en *tabla de propiedades / Bits de Datos*). Utiliza el componente *Separador* (carpeta de componentes *Wiring*) para separar los bits de los buses de entrada. Configura los *Separadores* con *Fanout 4* y *Bits De Entrada 4*. Conecta los bits obtenidos a los **MUX_4:1**. Utiliza otro *Separador* para juntar los bits que salen de los **MUX_4:1** y llevar el bus a la salida **Z**. Comprueba su funcionamiento.
- Vuelve a la hoja principal (doble clic en **main**) y añade otro componente. Nómbralo **MUX_4:1x16**. Coloca 4 **MUX_4:1x4**. Coloca las entradas (**x0**, **x1**, **x2**, **x3**, **s1** y **s0**) y una salida **Z**. Define **x0**, **x1**, **x2**, **x3** y **Z** como entradas/salidas de 16 bits (en *tabla de propiedades / Bits de Datos*). Utiliza el componente *Separador* (carpeta de componentes *Wiring*) para separar los bits de los buses de entrada. Configura los *Separadores* con *Fanout 4* y *Bits De Entrada 16*. Conecta las señales obtenidas a los **MUX_4:1x4**. Utiliza otro *Separador* para juntar los bits que salen de los **MUX_4:1x4** y llevar el bus a la salida **Z**. Comprueba su funcionamiento.
- Vuelve a la hoja principal y coloca un **MUX_4:1x16**. Añade un sumador de naturales (componente *Aritmética / Sumador*) y puertas XOR de 16 *Bits de Datos*. Finalmente añade otras puertas necesarias para el cálculo de los *flags*.
- Comprueba que la ALU funciona correctamente. Puedes utilizar el componente *Ver* para visualizar datos de los buses. Ten en cuenta que se puede cambiar la forma en la que el componente *Ver* interpreta el dato (*tabla de propiedades / Radix*).
- En el recurso Moodle “Entrega práctica 4” se explica el procedimiento a seguir para verificar el correcto funcionamiento de la ALU y su entrega.

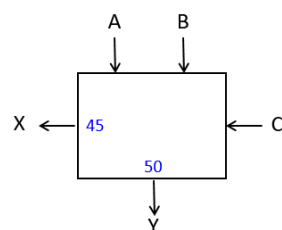
3 ANÁLISIS TEMPORAL

- Realiza el *análisis temporal* de cada uno de tus bloques: **MUX_4:1**, **MUX_4:1x4** y **MUX_4:1x16** (diseñados en el apartado 2.2) teniendo en cuenta que los retardos de las puertas utilizadas son: $d_{NOT} = 5$ ps; $d_{OR2-4} = 20$ ps; $d_{AND2-4} = 20$ ps; $d_{NOR16} = 55$ ps; $d_{XOR2} = 30$ ps ($1 \text{ ps} = 10^{-12} \text{ s}$).
- Realiza el análisis temporal del sumador, asumiendo que está diseñado con *full adders* de 5 puertas (2 XOR, 2 AND, 1 OR) y propagación de acarreo.
- Realiza el análisis temporal del todo el diseño. Para ello: 1) añade el *etiquetado temporal de los bloques*; 2) añade el *etiquetado temporal de las señales/nodos* con los retardos desde las entradas **A, B, T**. Ver ejemplos más abajo.
- Calcula la frecuencia máxima de cálculo expresada en número de Giga operaciones/segundo ($1 \text{ G} = 10^9$). Ten en cuenta que en cada operación pueden cambiar todas las entradas (**A, B, T**) y hay que esperar a que todas las salidas (**S, N, Z, V, C**) sean correctas para iniciar otro cálculo.
- Introduce los retardos obtenidos a través del recurso Moodle “Análisis temporal práctica 4”.

Etiquetado temporal de bloques

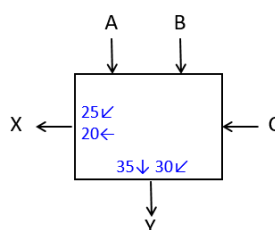
Etiquetamos cada salida del bloque con su retardo:

- si el retardo desde todas las entradas a esa salida es igual, la etiquetamos con dicho retardo;
- si hay distintos retardos de distintas entradas etiquetamos la salida con varios retardos, incluyendo flechas.



$$d_{A,B,C \rightarrow X} = 45 \text{ ps}$$

$$d_{A,B,C \rightarrow Y} = 50 \text{ ps}$$



$$d_{A,B \rightarrow X} = 25 \text{ ps}$$

$$d_{C \rightarrow X} = 20 \text{ ps}$$

$$d_{A,B \rightarrow Y} = 35 \text{ ps}$$

$$d_{C \rightarrow Y} = 30 \text{ ps}$$

Etiquetado temporal de señales

Etiquetamos cada señal con su **vector de retardos**: retardos máximos desde cada entrada del circuito a esa señal

Entradas: A, B, C, D

Salidas: W, X, Y, Z

$$d_{A,B,C \rightarrow W} = 40 \text{ ps}$$

$$d_{A,B,C \rightarrow X} = 85 \text{ ps}$$

$$d_{A,B,C \rightarrow Y} = 115 \text{ ps}$$

$$d_{D \rightarrow Y} = 20 \text{ ps}$$

$$d_{A,B,C \rightarrow Z} = 125 \text{ ps}$$

$$d_{D \rightarrow Z} = 30 \text{ ps}$$

